

3.1 - Computing on Data

• Computation

↳ A process that transforms one or more inputs to one or more outputs, following a well-defined "model", or "algorithm".

• Computation allows us to get what we want from data.

Data is not just numbers so we cannot just use addition, subtraction, multiplication, division.

* We can think of data as a matrix, and use linear algebra.

↳ It assumes "numbers."

Does it help us find beers made by Boston Beer?

Beers

name	brewer	alcohol
Sam Adams	Boston Beer	4.9
Goose IPA	Goose Island	5.9
Summer Ale	Boston Beer	5.3



name	brewer	alcohol
1	1	4.9
2	2	5.9
3	1	5.3

Computing on data depends on data...

• Relations

• Key-value pairs

• Documents

• Graphs

What purpose?

• Asking questions about the world we manage.

• Processing data ~~to~~ for preparing other datasets.

Examples of Data Computing Framework

Relational Data

↳ Relational Algebra: Foundation of SQL

↳ Datalog: Popular mechanism for inferencing with relational data.

Graph Data

↳ Graph is a well-studied data structure, before it became a database model.

↳ Finding neighbours, nodes matching paths, shortest path etc.

Key-Value data

↳ MapReduce

3.2 Relational Algebra

Algebra

- Mathematical system:
 - ↳ operands: Values to be operated upon (to generate new values)
 - ↳ operators: Symbols denoting operations that compute over operands and produce new values.
- Examples:
 - ↳ Arithmetic Algebra
 - operands: Numbers
 - operators: Addition, subtraction, multiplication, division
 - ↳ Boolean Algebra
 - operands: Truth values: false and true.
 - operators: And, or, not

Relational Algebra

- operands: Relations
- operators
 - ↳ What do we want to compute over one or ~~two~~ multiple relations?
 - ↳ We want to compute on relations to answer questions.

(15)

Relational Operators

- operators: relations as input, new relation as output.
- Basic operators:
 - Reduction: Make one table smaller.
 - selection $R_1 \rightarrow [\sigma_p] \rightarrow R_2$
 - Projection
 - Combination: Combine two tables.
 - set operations
 - Union, difference.
 - Cartesian Product
 - Renaming: change Attribute names.
- Derived operations
 - Intersection, complement
 - Joins (natural, equi-join, theta join, semi-join)

3.3 Relational Algebra - Basic Operators 1

* Operators Operate upon operands

- Relational Algebra computes on relations
- We refer to a relation as operand by its name
- A relation name is an operand, and a RA expression.
- What is the simplest relational algebra expression?

Basic Operators

- Reduction: Make a table smaller
 - selection σ
 - Projection π
- Combination: Combine two tables
 - Set Union $\cup$
 - Set Difference $-$
 - Cartesian Product $\times$
- Renaming ρ : Change Attribute names.

3.4 Relational Algebra

Basic Operators 2

Reduction Operators

- Reducing rows: Selection: σ
- Reducing columns: Projection: π

Selection σ

- Notation: $\sigma_c(R)$
 $\rightarrow$ condition
- Input: relation R
- Parameters: c is a condition: $=, <, >$, and, or, not.
- Output:
 - A relation as a subset of R that satisfies condition c .
 - Schema: same as R
- What is the major of Bugs Bunny?
 - $\sigma_{name = \text{"Bugs Bunny"}}(\text{Students})$
 - $\sigma_{price < 5}(\text{Sells})$

Projection π

(16)

- Notation: $\pi_{A_1, \dots, A_n}(R)$
- Input: relation $R(B_1, \dots, B_m)$
- Parameters: $\{A_1, \dots, A_n\} \subseteq \{B_1, \dots, B_m\}$
- Output:
 - A relation of tuples with attribute $A_1, \dots, A_n$ from R .
 - Duplicates are removed.
 - Schema: $\{A_1, \dots, A_n\}$
- What majors do students have?
 - $\pi_{majors}(\text{Students})$

3.5 Relational Algebra

Basic Operators 3

Combination Operators

- Combining two relation R_1 and R_2
- Set operations
 - Union: $R_1 \cup R_2$ (addition)
 - Difference: $R_1 - R_2$ (subtraction)
- Cartesian Product (multiplication)
 - $R_1 \times R_2$

Set Union

- Notation: $R_1 \cup R_2$
- Input: R_1 and R_2 , which have the same schema.
- Output:
 - A relation with all tuples in R_1 and R_2 (no duplicates)
 - Schema: same as R_1 and R_2 .
- What are all the people in the Academic World?
 $\pi_{name}(\text{Students}) \cup \pi_{name}(\text{Professors})$

Set Difference

- Notation: $R_1 - R_2$
- Input: R_1 and R_2 , which have the same schema.
- Output:
 - A relation with all tuples in R_1 but not in R_2
 - Schema: same as R_1 and R_2 .
- What students took CS411 but not CS412?
 - $\pi_{name}(\sigma_{number='CS411'} Enrolls) - \pi_{name}(\sigma_{number='CS412'} Enrolls)$

Cartesian Product

- Notation: $R_1 \times R_2$
- Input: $R_1(A_1, \dots, A_n), R_2(B_1, \dots, B_m)$
- Output:
 - A relation consisting of all pairs of each tuple from R_1 and each tuple from R_2 .
 - schema $(A_1, \dots, A_n, B_1, \dots, B_m)$
 - Assume no attribute clash. Or, use $R_1.A$ vs. $R_2.A$ to distinguish

- Useful for collecting data into one table
SE = students $\times$ Enrolls

What courses did Bugs Bunny take?

BB := $\sigma_{s.id = E.id \text{ AND name} = \text{"Bugs Bunny"}}(SE)$

Answer := $\pi_{number}(BB) = \{(411), (426)\}$

Overall:

$\pi_{number}(\sigma_{s.id = E.id \text{ AND name} = \text{"Bugs Bunny"}}(students \times Enrolls))$

⁽¹⁷⁾
Drinkers (drinker, dAddr, hobby, frequent) ^(Drinkers)

3.6 Relational Algebra

Basic operator for renaming schema

Renaming

• Notation: $\rho_S(B_1, \dots, B_n)(R)$

• Input: $R(A_1, \dots, A_n)$

• Output:

- The same relation instance
- Schema $S(B_1, \dots, B_n)$

• Why?

- so that names are more meaningful.

• $\rho_{(student, advisor)}(\pi_{name, advisor} students)$

• or: $\rho_{name \rightarrow student}(\pi_{name, advisor} students)$

- So that names do not clash.

• $s.id \rightarrow student (students) \times p.id \rightarrow professor (Professors)$

3.7 Relational Algebra

Derived Operators

Derived Operations

- Some operations are very often used.
- But they can be expressed in terms of basic operations.
- We will introduce:

• Set Intersection $R_1 \cap R_2$

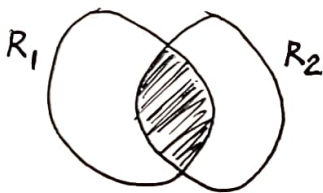
• Theta Join $R_1 \bowtie_{\theta} R_2$

• Natural Join $R_1 \bowtie R_2$

Set Intersection

- Notation: $R_1 \cap R_2$
- Input: R_1 and R_2 , which have same schema.
- Output:
 - A relation with all tuples common in R_1 and R_2 .
 - Schema: same as R_1 and R_2
- Which students take both 411 and 426?
 - $\pi_{id}(\sigma_{\text{number}="411"} \text{Enrolls}) \cap \pi_{id}(\sigma_{\text{number}="426"} \text{Enrolls})$
 $= \{(1), (4)\} \cap \{(1), (2)\} = \{(1)\}$
- Intersection can be derived from Difference

$$R_1 \cap R_2 = R_1 - (R_1 - R_2)$$



~~Diagram~~

Theta Join

Notation: $R_1 \bowtie_{\theta} R_2$, where θ is a condition.

Input: $R_1(A_1, \dots, A_n), R_2(B_1, \dots, B_m)$

Parameters: θ is a condition over R_1 and/or R_2 's attributes.

Output:

- A relation of the product of R_1 and R_2 filtered by condition θ .
- I.e. can be expressed by $\sigma_{\theta}(R_1 \times R_2)$
- Schema: $(A_1, \dots, A_n, B_1, \dots, B_m)$
 - Assume no attribute name clash, or use $R_1.A$ vs $R_2.A$ to distinguish.

What courses did Bugs Bunny take?

- $\pi_{\text{number}} \sigma_{s.id = E.id \text{ AND name} = \text{"Bugs Bunny"}} (\text{Students} \times \text{Enrolls})$
- $\pi_{\text{number}} (\text{Students} \bowtie_{s.id = E.id \text{ AND name} = \text{"Bugs Bunny"}} \text{Enrolls})$

Natural Join (θ is natural)

- Notation: $R_1 \bowtie R_2$
- Input: $R_1(A_1, \dots, A_n), R_2(B_1, \dots, B_m)$
- Output:
 - A relation combining all pairs of tuples in R_1 and R_2 that agree on their "common" attributes
 $\{A_1, \dots, A_n\} \cap \{B_1, \dots, B_m\}$, the join attributes.
 - I.e. can be expressed by $\sigma_{\theta}(R_1 \times R_2)$
 - Schema: $\{A_1, \dots, A_n\} \cup \{B_1, \dots, B_m\}$, ~~here,~~
i.e., merging the two schemas and removing duplicates.
- What causes did Bugs Bunny take?
 $\pi_{\text{number}}(\text{Students} \bowtie_{s.id = E.id \text{ AND name} = \text{"Bugs Bunny"} \text{ Enrolls})$
 $\pi_{\text{number}}(\sigma_{\text{name} = \text{"Bugs Bunny"}} \text{ Students} \bowtie \text{Enrolls})$

3.8 MapReduce for Key-Value (19)

Data

MapReduce

- A programming model for processing key-value data.
- For a simplified data processing framework over large clusters.
- Ex. Apache Hadoop.

Map and Reduce in Functional Programming

- Map:
 - Input: Function M , list $L = [e_1, \dots, e_n]$
 - Output: map M $L = [M(e_1), \dots, M(e_n)]$
 - Apply ^{given} function to every element of the list.
 - E.g., map square $[1, 2, 3, 4, 5] = [1, 4, 9, 16, 25]$
- Reduce:
 - Input: Function R , list $L = [e_1, \dots, e_n]$
 - Output: reduce R $L = R(e_1, \dots, e_n)$.
 - Apply given function to aggregate all elements of the list.
 - E.g. reduce sum $[1, 4, 9, 16, 25] = 55$

Computing on K-V Data with MapReduce

- database $DB = [e_1, \dots, e_n]$, where $e_i = (k_i, v_i)$, a key-value pair.
- Execute M and R in MapReduce system with steps.

Map $\rightarrow$ Group $\rightarrow$ Reduce

- Map on each $e_i = (k_i, v_i)$ with M .
 - $M(k_i, v_i) \rightarrow [(k_{i1}, v_{i1}), \dots, (k_{im}, v_{im})]$
- Group
 - Organize (k_{ij}, v_{ij}) pairs by key k_{ij} -- each group is $(k_{ij}, [\text{values with key } k_{ij}])$
- Reduce on each group $(k_{ij}, [\text{values}])$ by reduce function R .
 - Output: $[k_{ij}, R(\text{values with } k_{ij})]$

Relational Operation with MapReduce

- $\sigma_{\text{brewer} = \text{"Boston Beer"}} \text{Beers}$
- function $M(k, v)$: if $v = \text{"Boston Beer"}$ emit (k, v)
- function $R(k, v)$: emit (k, v)